# קטע קוד ראשון

```
 1. template<class E> class SmartPtr {
 2.   class Data {
 3.     friend class SmartPtr<E>; // required for line 37
 4.     SmartPtr<E>* const ptr;
 5.     template<class Param> // Param can be either Data or E
 6.     const Data& assign(const Param& param) {
 7.       if(*(ptr->refCounter) == 1) { *(ptr->pdata) = param; }
 8.       else { ptr->copyOnWrite(param); }
 9.       return *this;
10.     }
11.   public:
12.     Data(SmartPtr<E>* p): ptr(p) {}
13.     const Data& operator=(const E& e) { return assign(e); }
14.     const Data& operator=(const Data& d) { return assign(d); }
15.     operator const E&()const { return *ptr->pdata; }
16.   }; // end of class Data
17.
18.   E* pdata = nullptr;
19.   int* refCounter = nullptr;
20.
21.   void attach(const SmartPtr<E>& a) {
22.     pdata = a.pdata;
23.     refCounter = a.refCounter;
24.     ++(*refCounter);
25.   }
26.   void detach() {
27.     if(refCounter && --(*refCounter) == 0) {
28.       delete refCounter;
29.       cout << "detach, deleting: " << (void*)pdata << endl;
30.       delete pdata;
31.     }
32.   }
33.   void copyOnWrite(const E& e) {
34.     detach(); attach(e);
35.   }
36.   void copyOnWrite(const Data& data) {
37.     detach(); attach(*(data.ptr));
38.   }
39. public:
40.   SmartPtr(const E& data)
41.     : pdata(new E(data)), refCounter(new int(1)) {
42.     cout << "SmartPtr born, address: " << (void*)pdata << endl;
43.   }
44.   SmartPtr(const SmartPtr& p) { attach(p); }
45.   SmartPtr& operator=(const SmartPtr& ptr) {
46.     detach(); attach(ptr); return *this;
47.   }
48.   ~SmartPtr() { detach(); }
49.   Data operator*() { return Data(this); }
50.   const E& operator*()const { return *pdata; }
51.   operator const E*()const { return pdata; }
52. };
```

<u>נתון</u>: הקוד תקין וניתן לכתוב עבורו main תקין שיעשה שימוש בכל הפונקציות שהוגדרו.
להלן main אפשריים לקוד שלמעלה (כל main עומד בפני עצמו):

```
// M A I N - 1
int main() {
  SmartPtr<int> ptr1 = 30;
  auto ptr2 = ptr1;
  *ptr1 = 10;
  cout << *ptr1 << endl << *ptr2 << endl;
}
```

```
// M A I N - 2
int main() {
  std::list<SmartPtr<std::string>> v1;
  v1.push_back({"Hello"});
  v1.push_back(std::string("world"));
  v1.push_back(SmartPtr<std::string>("!!!"));
  auto v2 = v1; // std::list copy ctor
  **v2.begin() = {"world"};
  // *(++v1.begin()) = {"hello"};
  for(const auto& pv : v1) {
    cout << *pv << endl;
  }
}
```

```
// M A I N - 3
int main() {
  SmartPtr<int> ptr1 = 10;
  auto ptr2 = ptr1;
  *ptr1 = *ptr1;
  *ptr1 = (int)*ptr1;
  ptr1 = ptr1;
  cout << *ptr1 << ' ' << *ptr2 << endl;
}
```

## קטע קוד B

```
 1. template<class Key, class Value>
 2. class MultiMap {
 3.   vector<std::pair<const Key, list<Value>>> mmap;
 4. public:
 5.   void add(const Key& k, const Value& v) {
 6.     auto itr = mmap.begin();
 7.     auto end = mmap.end();
 8.     for( ; itr != end && itr->first != k; ++itr);
 9.     if(itr != end) {
10.       itr->second.push_back(v);
11.     }
12.     else {
13.       mmap.push_back({k, {v}});
14.     }
15.   }
16.   auto begin() { return mmap.begin(); }
17.   auto end() { return mmap.end(); }
18.
19.   class Keys {
20.    const MultiMap* myMap;
21.    using baseIterator = typename // =>
22.      vector<std::pair<const Key, list<Value>>>::const_iterator;
23.   public:
24.     Keys(const MultiMap* mMap): myMap(mMap) {}
25.     struct iterator: public baseIterator {
26.       iterator(baseIterator itr) : baseIterator(itr) {}
27.       const Key& operator*() const {
28.         return (*this)->first;
29.       }
30.     };
31.     iterator begin()const { return myMap->mmap.cbegin(); }
32.     iterator end()const { return myMap->mmap.cend(); }
33.   };
34.   Keys keys() const {
35.     return Keys(this);
36.   }
37. };
38.
39. int main() {
40.   MultiMap<int, std::string> m;
41.   m.add(7, "Seven");
42.   m.add(7, "Sheva");
43.   m.add(7, "Sabbaa");
44.   m.add(70, "Seventy");
45.   for(auto& p : m) {
46.     cout << p.first << ": " << p.second.size() << endl;
47.   }
48.   for(auto& k : m.keys()) {
49.     cout << k << endl;
50.   }
51. }
```

## קוד C – לשאלות רשות

```
 1. class MyString {
 2.   int size;
 3.   char* str = nullptr;
 4.
 5.   // private helpers
 6.   void stealFrom(MyString&& s) {
 7.     size = s.size;
 8.     std::swap(str, s.str);
 9.   }
10.   MyString(size_t len): size(len), str(new char[size + 1]) {
11.     str[0] = '\0';
12.   }
13.
14. public:
15.   MyString(const char* s = "")
16.     : MyString(strlen(s)) { strcpy(str, s); }
17.   ~MyString() { cout << "dtor" << endl; delete[]str; }
18.   MyString(MyString&& s) { stealFrom(std::move(s)); }
19.
20.   // note: operator= gets MyString by value - in purpose!
21.   MyString& operator=(MyString s) {
22.     cout << "in operator=" << endl;
23.     stealFrom(std::move(s));
24.     return *this;
25.   }
26.   friend MyString operator+(const MyString& l, const MyString& r) {
27.     MyString s(l.size + r.size);
28.     strcpy(s.str, l.str);
29.     strcpy(s.str + l.size, r.str);
30.     return s;
31.   }
32.   friend ostream& operator<<(ostream& out, const MyString& s) {
33.     return out << (s.str? s.str : "(null)") << " ("
34.                << &s << " char* = " << (void*)s.str << ')';
35.   }
36. };
37.
38. int main() {
39.   MyString s1 = "Hello";
40.   cout << s1 + " world!" << endl;
41.   MyString s2;
42.   cout << "s2: " << s2 << endl;
43.   s2 = "world " + s1;
44.   cout << "s2: " << s2 << endl;
45. }
```

**בהצלחה !**